

Understanding SDT Markers and Debugging with Subtlety

Discover how to use Systemtap Dtrace style markers, which can be a great aid in a production environment to monitor the program flow.



Debugging has become integral to a programmer's life. In C and C++, we normally debug using `printf()` / `couts`, `gdb`, `valgrind`, etc. SDT markers are very handy due to their simplicity yet great debugging capabilities.

What are SDT markers?

The abbreviation stands for 'Systemtap Dtrace' style markers. They are statically placed user space markers which can later be probed *dynamically* for some useful information.

Why use them?

SDT markers are normally placed at important spots in programs and can later be probed for some useful information; for instance, if you want to determine the code flow, the values of some important variables, etc. This information is what we look for while debugging.

SDT markers are being quite widely used now. For example, applications/libraries like PostgreSQL, MySQL,

Mozilla, Perl, Python, Java, Ruby, libvirt, QEMU, glib, etc have these markers embedded in them. Most of us aren't even aware of them but a quick scan of a standard Fedora 20 system will show a total of over 4000 SDT markers. However, this may not be true for other distros.

How to use SDT markers

Using SDT markers is very simple. You have to include the `<sys/sdt.h>` header in your program. If this header file is absent, then install the `systemtap-sdt-devel` package for Fedora, RedHat and related distros. For distros like Ubuntu and Debian, the package name is `systemtap-sdt-dev`. After including the header file, you just have to use a variant of the macro `STAP_PROBE(provider_name, marker_name)` wherever you want the marker to be placed.

The arguments `provider_name` and `marker_name` are very important. They establish the identity of the marker. Here is a simple example to show how SDT markers are used: